



LiDAR SLAM with Multiple Object Tracking(MOT) for Autonomous Driving

ME5400 Project

April 13, 2026

Contents

1	Introduction and Background	1
2	Related Work Analysis	2
3	Overall System Architecture, PointPillars / MCTrack, and Data Flow	4
3.1	FAST-LIO Front-End	5
3.2	PointPillars Detection Module	7
3.3	MCTrack Module and Data-Flow Details	8
4	Engineering Implementation and System Integration	10
4.1	Overall Code Organization	10
4.2	Online System Design	10
4.3	Result Archiving and the Reproducible Experimental Pipeline	12
5	Dynamic Weight Optimization	12
5.1	Design Motivation and Code Location	12
5.2	Weight Computation	13
5.3	How the Weight Is Injected into FAST-LIO	13
6	Joint Backend Optimization	14
6.1	The Ego-Only Sliding-Window Problem	14
6.2	Residual Modeling and the Gating Strategy	14
6.3	Value and Limitations of the Joint Backend	15
7	Experimental Analysis	16
7.1	Experimental Setup and Result Sources	16
7.2	Online Experiments: Aggregate Results	16
7.3	Integrated Analysis and Failure Modes	18
8	Conclusion and Future Plan	18
	Appendix: All Online Experimental Figures	21

Abstract

This report provides a systematic account of the ME5400 course project, organized into introduction and background, system architecture, code design, dynamic-weight optimization, joint backend optimization, experimental analysis, and conclusions with future work. The project uses FAST-LIO as the LiDAR-inertial localization front-end, PointPillars for 3D object detection, and MCTrack for multi-object tracking and velocity estimation. The tracking results are fed back to FAST-LIO for adaptive down-weighting of dynamic points and are then passed to an ego-only joint backend to refine the ego trajectory. In addition to retaining the original system background and implementation details, the report consolidates the current online experimental results under Results/, covering seven sequences (0003, 0006, 0007, 0009, 0010, 0013, and 0020), with all online result figures included in the appendix. The results show that the most stable gain comes from the front-end closed loop formed by “FAST-LIO pose aiding MCTrack + MCTrack feeding back dynamic weights”: in online mode, FAST-LIO+MCTrack outperforms the baseline on all seven sequences, with an average ATE RMSE improvement of 18.00%. Although the current ego-only joint backend can further suppress global drift on some sequences, its stability remains weaker than that of the front-end loop itself. Overall, the system has formed a runnable, replayable, visualizable, and quantitatively evaluable engineering validation pipeline, while also providing a directly extensible basis for future object-centric LiDAR-inertial joint optimization.

Keywords: FAST-LIO; PointPillars; MCTrack; dynamic-scene SLAM; multi-object tracking; joint optimization; ROS

1 Introduction and Background

Localization and perception in dynamic road scenes involve a fundamentally conflicting set of requirements. For LiDAR SLAM or LiDAR-inertial odometry, the system prefers most environmental geometry to be static, repeatedly observable, and reusable over time. For autonomous driving or mobile robotics perception, however, vehicles, pedestrians, and cyclists on the road are precisely the most important dynamic targets. These targets create time-varying geometric structures in the point cloud. If they are directly treated as static points and written into the map and registration residuals, the system suffers from ghosting, local mismatches, and accumulated drift. If they are removed indiscriminately, potentially useful geometric information is discarded as well, leading to insufficient constraints in scenes with sparse static structure, drastic viewpoint change, or high-speed motion.

Multi-object tracking (MOT) faces the opposite but equally difficult problem. Cross-frame association of 3D bounding boxes depends heavily on stable ego-motion compensation. If the

tracker cannot obtain sufficiently accurate and high-frequency ego poses, the geometric relationships between detections are corrupted by ego motion, leading to broken trajectories, unstable velocity estimates, and ID switches. As a result, localization and tracking are not merely cascaded modules; they are naturally bidirectionally dependent. The more accurate the localization, the more stable the tracking. The more stable the tracking, the better the system can identify which points come from moving targets, thereby feeding back into mapping and localization.

This project follows exactly this idea. Rather than mechanically concatenating FAST-LIO, PointPillars, and MCTrack, it builds a collaborative closed-loop pipeline around the principle that localization should assist tracking and tracking should in turn improve localization. The original KITTI Tracking rosbag provides LiDAR and IMU data. PointPillars outputs 3D detections. FAST-LIO produces optimized poses and IMU-predicted poses. MCTrack combines poses and detections to perform online tracking. The tracking outputs are then fed back to FAST-LIO for dynamic-point down-weighting and are also sent to a joint backend to refine the ego trajectory. In this way, dynamic targets are no longer treated only as noise to be filtered out; they begin to participate in front-end weight modulation and backend optimization.

In a broader research context, recent work on dynamic-scene SLAM / SLAMMOT has generally moved toward sharing information between localization and tracking. However, most methods still focus on dynamic-point filtering, object-state estimation, or implicit data association, and only a few truly convert dynamic objects into geometric references that help ego localization. The current version of this project is still an engineering-oriented, loosely coupled prototype rather than a final tightly coupled factor-graph system. Even so, it has already completed a full information loop, unified script-based management, visual verification, and multi-sequence result archiving. It therefore has clear value as a course project and also provides a practical basis for further extension toward object-centric joint optimization.

2 Related Work Analysis

To position this project within the current research landscape, this report further selects five recent representative works summarized in `slammot_comparison.md`: Conf SLAMMOT (*LiDAR SLAMMOT based on Confidence-guided Data Association*), LIMOT (*A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking*), IMM-SLAMMOT (*Visual SLAMMOT Considering Multiple Motion Models*), Online Dynamic SLAM (*Online Dynamic SLAM with Incremental Smoothing and Mapping*), and DL-SLOT (*Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization*) [5, 6, 7, 8, 9]. These works cover LiDAR-only, LiDAR+IMU, and stereo-vision sensing pipelines, as well as typical implementation routes such as sliding-window graph optimization, bundle adjustment, and iSAM2-based incremental optimization.

Table 1: Representative recent SLAMMOT systems in dynamic scenes and the positioning of this project

Work	Sensors and front-end	Coupling / optimization	Key characteristics and implications for this project
Conf SLAM-MOT	LiDAR; LiDAR odometry front-end	Tightly coupled; confidence-guided data association embedded in graph optimization	Emphasizes directly incorporating detection / association confidence into joint optimization, which is instructive for future weighting of object factors in this project.
LIMOT	LiDAR+IMU; custom LIO front-end	Tightly coupled; sliding-window factor graph	Represents the mainstream direction of unified modeling for LiDAR-inertial odometry and MOT, and is especially relevant for moving this project from topic-level loose coupling toward unified state optimization.
IMM-SLAMMOT	Stereo vision; ORB-SLAM2 front-end	Tightly coupled; joint BA + multiple motion models	Highlights the importance of model switching for complex dynamic targets, showing that a single constant-velocity assumption is insufficient for real traffic scenes.
Online Dynamic SLAM	Stereo vision; custom VO front-end	Tightly coupled; iSAM2 incremental smoothing and mapping	Focuses on real-time optimization in on-line dynamic scenes, indicating that incremental backends are critical for scalable real-time systems.
DL-SLOT	LiDAR; LiDAR odometry front-end	Tightly coupled; collaborative graph optimization / sliding-window optimization	Reflects the trend of sharing window-level information among detection, tracking, and localization in LiDAR pipelines, offering direct insights for window management and marginalization in this project.

Three observations can be drawn from these works. First, **system architectures are rapidly evolving from cascaded pipelines toward tightly coupled graph optimization**. These five papers almost never treat SLAM and MOT as fully independent black-box modules. Instead, they share poses, object states, association confidence, or motion models so that front-end and backend components can refine one another within the same optimization window. Second, **the use of dynamic targets still mainly centers on filtering, association, and joint estimation**. Existing methods have started to incorporate target velocity, trajectory length, and detection confidence, but most still operate at the level of dynamic-point removal, dynamic-state estimation, or implicit association weighting. They have not yet fully converted the rigid-body geometry of dynamic vehicles into explicit LiDAR-inertial localization constraints. Third, **real-time capability and engineering deployability have become important criteria for evaluating system value**. Whether through the sliding-window schemes in LIMOT and

DL-SLOT or the iSAM2-based incremental pipeline in Online Dynamic SLAM, the message is consistent: research systems cannot focus on accuracy alone, but must also account for online execution, window management, and scalable deployment.

Compared with these works, the current project can be positioned as follows: the engineering closed loop is already operational, but unified optimization is still at the prototype stage. Its strengths are clear. The FAST-LIO front-end is mature and stable. PointPillars and MC-Track have already been fully integrated into ROS. The dynamic-weight closed loop has been verified on multiple sequences. Its limitations are equally clear. SLAM and MOT still interact mainly through topics rather than a shared state graph, and the joint backend is still ego-only. In other words, the project already has the infrastructure needed to evolve into a tightly coupled SLAMMOT system. The most valuable next step is to write MCTrack confidence, velocity, and trajectory stability more directly into a unified factor graph and to explore how rigid-body constraints from dynamic targets can convert a “source of disturbance” into a “moving landmark.”

3 Overall System Architecture, PointPillars / MCTrack, and Data Flow

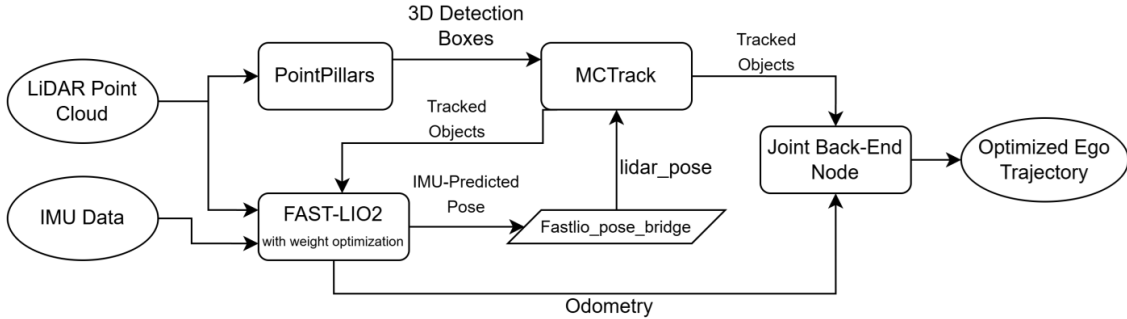


Figure 1: Overall system architecture. It can be interpreted as a three-layer process of perception first, tracking alignment next, and joint correction last.

In more intuitive terms, the system resembles a production line with clearly separated responsibilities. The first layer is responsible for perceiving the environment. FAST-LIO estimates how the ego vehicle moves from LiDAR and IMU measurements, while PointPillars identifies surrounding objects from the point cloud. The middle layer aligns and links these information streams. `fastlio_pose_bridge.py` converts FAST-LIO poses into `/mctrack/lidar_pose`, which MCTrack can directly use. MCTrack then combines these poses with detections from `/detection/lidar_detections` to determine which vehicle is which and where each one is moving, finally outputting `/mctrack/tracked_objects`. The final layer per-

forms integrated correction. `joint_backend_ego_node.py` receives both `/Odometry` and `/mctrack/tracked_objects`, refines the ego trajectory in a sliding window, and outputs the more stable `/joint_backend/odom`.

If the system is understood from the data-flow perspective, the process can be summarized in seven steps:

1. Sensors first acquire raw LiDAR point clouds and IMU data, and feed them simultaneously to FAST-LIO and PointPillars.
2. PointPillars produces the 3D detections of the current frame, indicating which targets are present nearby.
3. FAST-LIO outputs the current ego pose and continuously provides higher-frequency motion predictions.
4. The pose bridge converts these localization outputs into the format directly consumable by the tracking module.
5. MCTrack combines “what was observed” with “how the ego vehicle moved” to complete object association, trajectory updates, and velocity estimation.
6. Tracking outputs are fed back to FAST-LIO so that points likely belonging to dynamic objects can be down-weighted.
7. The same tracking outputs are also sent to the joint backend to further refine the ego trajectory.

3.1 FAST-LIO Front-End

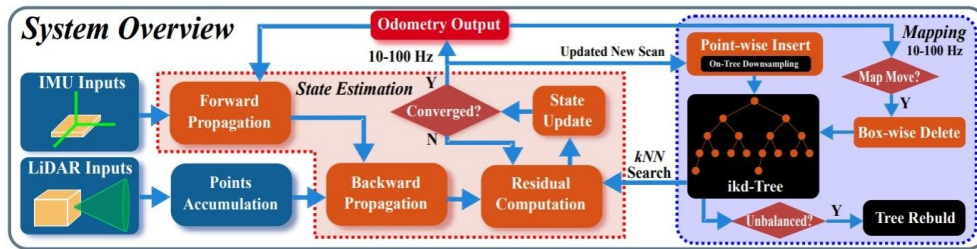


Figure 2: FAST-LIO2 front-end architecture. In this project, object information from MCTrack is injected into the point-cloud preprocessing pipeline to realize adaptive down-weighting of dynamic points.

FAST-LIO is the localization backbone of the entire system and the starting point of the data flow described above. It directly consumes LiDAR point clouds and IMU data from the KITTI rosbag and internally uses a tightly coupled LiDAR-inertial estimation pipeline to output stable

ego poses. In this project, FAST-LIO not only generates globally consistent /Odometry, but also publishes /Odometry_imu_predicted as a high-frequency predicted pose, providing a unified reference for later tracking and backend optimization.

From the perspective of module responsibilities, FAST-LIO plays three roles in the system. First, it provides MCTrack with reliable ego poses, allowing 3D detections to be compensated for ego motion before cross-frame association. Second, it supplies the joint backend with raw odometry constraints so that backend optimization does not drift too far from the front-end estimate. Third, through preprocess.cpp, it receives /mctrack/tracked_objects and converts target velocity, acceleration, score, and bounding-box information into point-level weights, thereby realizing a front-end closed loop in which tracking feeds back into localization.

From an algorithmic viewpoint, the FAST-LIO front-end can be understood as an iterative filter combining IMU propagation with LiDAR geometric correction. Let the system state be $\mathbf{x}_k$ and the IMU input be $\mathbf{u}_k$. The prediction step can be written as

$$\mathbf{x}_{k+1|k} = f(\mathbf{x}_{k|k}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_{k|k} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (1)$$

where $\mathbf{P}$ is the state covariance and $\mathbf{F}_k$ is the linearized state-transition matrix. For the j -th point in the current frame, FAST-LIO further constructs a point-to-plane geometric residual using a reference plane in the local map:

$$r_j = \mathbf{n}_j^\top (\mathbf{R}(\mathbf{x}) \mathbf{p}_j + \mathbf{t}(\mathbf{x}) - \mathbf{q}_j), \quad (2)$$

and minimizes through iterative updates

$$\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} \left(\sum_j \|r_j\|_{\square_j}^2 + \|\delta \mathbf{x}\|_{\mathbf{P}_{k+1|k}^{-1}}^2 \right). \quad (3)$$

These equations describe the core role of FAST-LIO in this project. The IMU provides high-frequency continuous prediction, while LiDAR point-cloud residuals pull the pose back toward a geometrically consistent solution. This is why FAST-LIO can not only stably output /Odometry, but also serve as the spatiotemporal reference for MCTrack.

To feed FAST-LIO outputs into the tracking module, the project additionally implements fastlio_pose_bridge.py. This bridge node converts /Odometry into /mctrack/lidar_pose, which MCTrack can consume directly. It therefore transforms an odometry message originally used mainly for localization into a unified pose input for the detection-and-tracking chain. As a result, FAST-LIO in this system should not be viewed merely as an independent odometry front-end, but as the spatiotemporal reference center of the whole closed-loop system. The dynamic-weight optimization discussed later is, in essence, an enhancement built on top of this front-end.

3.2 PointPillars Detection Module

PointPillars [2] is wrapped as a ROS node through `MMDet3D/local/ros/nodes/kitti_pointpillars_bag_` allowing it to directly consume KITTI point clouds from rosbag. Its role is not simply to output raw detections, but to provide the entire system with unified, traceable, and time-stamped 3D detection messages. The project defines a unified message format in `catkin_ws/src/ME5400/msg/Detection3D.msg` and `Detection3DArray.msg`, including frame ID, category, confidence, 2D box, 3D dimensions, LiDAR-frame position, and yaw angle. In this way, PointPillars and MCTrack do not depend on any private third-party message format; they communicate through project-defined message interfaces.

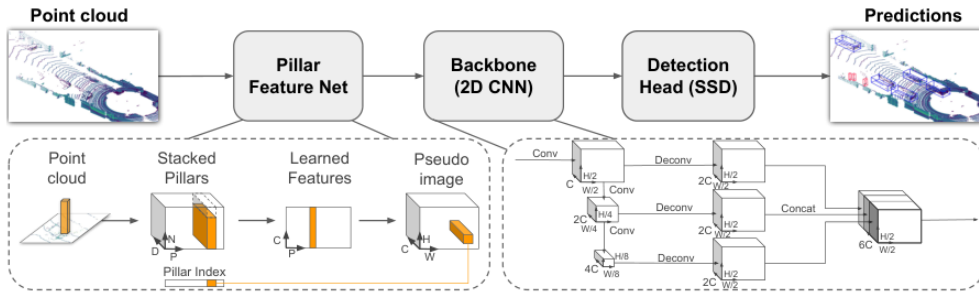


Figure 3: Network architecture from the original PointPillars paper. The network consists of a Pillar Feature Net, a 2D CNN backbone, and an SSD detection head. The image is reproduced from the CVPR 2019 paper by Lang *et al.* [2].

From the original paper design, the essence of PointPillars is not merely BEV detection. It first partitions sparse point clouds into vertical pillars, then uses the Pillar Feature Net to encode the points in each pillar into learned features, scatters these features into a pseudo-image, and finally applies a 2D CNN backbone and SSD head for object-box regression and classification. This design transforms a costly 3D convolution problem into a 2D convolution problem, thereby balancing speed and accuracy on KITTI. For the same reason, it is highly suitable as the real-time 3D detection front-end in this project’s online system.

Written in equations, if the l -th pillar contains the point set $\mathcal{P}_l = \{\mathbf{p}_i\}$, PointPillars first constructs an augmented feature for each point:

$$\tilde{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - \bar{x}_l, y_i - \bar{y}_l, z_i - \bar{z}_l, x_i - x_l^c, y_i - y_l^c], \quad (4)$$

where $(\bar{x}_l, \bar{y}_l, \bar{z}_l)$ is the mean of points inside the pillar and (x_l^c, y_l^c) is the pillar center. After the Pillar Feature Net, the pillar-level feature can be written as

$$\mathbf{f}_l = \max_{i \in \mathcal{P}_l} \phi(\tilde{\mathbf{p}}_i), \quad (5)$$

which is then scattered into the BEV pseudo-image:

$$\mathbf{I}(u, v) = \begin{cases} \mathbf{f}_l, & (u, v) = \text{index}(l), \\ \mathbf{0}, & \text{otherwise,} \end{cases} \quad (6)$$

Finally, the detection head jointly optimizes classification, box regression, and orientation prediction:

$$\mathcal{L}_{\text{det}} = \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (7)$$

This also explains why PointPillars fits this project well: point clouds are first compressed into a regular BEV representation, after which efficient 2D convolutions are sufficient to produce 3D detections, satisfying the requirements of online inference and real-time ROS publishing.

From an engineering perspective, this design serves three purposes. First, the node reads KITTI calibration information to ensure consistency between detection outputs and coordinate transforms. Second, it keeps a relatively low detection-confidence threshold so that more candidate boxes are handled later by the tracker and gating strategy instead of being pruned too early at the detector stage. Third, the unified message structure allows the same detection outputs to support MCTrack, visualization, evaluation, and result replay.

3.3 MCTrack Module and Data-Flow Details

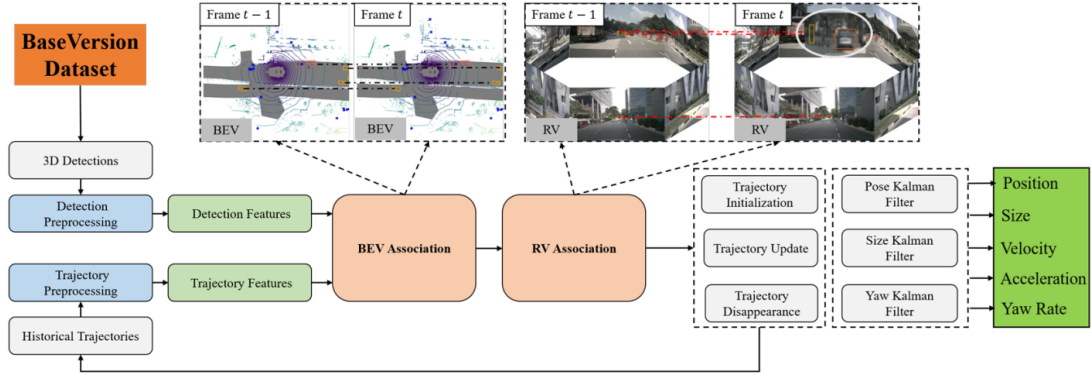


Figure 4: Schematic of the MCTrack 3D MOT framework. Its outputs contain not only position and size, but also velocity, acceleration, trajectory length, and related motion attributes.

The online MCTrack node is implemented in `catkin_ws/src/ME5400/scripts/mctrack_online_node.py`. It subscribes by default to `/mctrack/lidar_pose` and `/detection/lidar_detections`, maintains an internal pose buffer and a queue of pending detections, and processes detections only after a pose with a suitable timestamp is available. This “detections wait for pose, pose drives processing” design avoids the temporal misalignment that arises when messages are processed strictly in order of arrival.

MCTrack publishes both `/mctrack/markers` for visualization and `/mctrack/tracked_objects` for downstream optimization. In `TrackedObject.msg`, in addition to object pose and size, the message records `track_length`, `velocity`, `velocity_fusion`, `velocity_diff`, `velocity_curve`, and `acceleration`. These fields are crucial to this project because the subsequent dynamic-weight optimization and joint-backend gating depend on motion attributes rather than only on box position.

Under the current `kitti_fastlio.yaml` configuration, the main state estimate of MC-Track in online mode can be approximated as a 2D constant-velocity Kalman filter. If the trajectory state is

$$\mathbf{x}_k = [p_x, p_y, v_x, v_y]^\top, \quad (8)$$

its prediction model is

$$\mathbf{x}_{k|k-1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{k-1|k-1}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_{k|k-1} + \mathbf{v}_k, \quad (9)$$

followed by the standard Kalman update

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R})^{-1}, \quad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \mathbf{x}_{k|k-1}). \quad (10)$$

At the data-association level, the current online configuration mainly uses a 3D cost matrix based on predicted boxes together with Hungarian matching:

$$\mathcal{A}^* = \arg \min_{\mathcal{A} \in \Pi} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad C_{ij} = 1 - \text{RO-GDIOU}_{3D}(\hat{B}_i, B_j), \quad (11)$$

where $\hat{B}_i$ is the predicted track box and B_j is the current detection box. In other words, FAST-LIO first stabilizes the cross-frame geometry through ego pose, after which MCTrack converts discrete detections into continuous trajectories through motion prediction and global optimal matching. This is the foundation on which later velocity estimation, dynamic weights, and the joint backend become possible.

Therefore, one key point of this section is that PointPillars and MCTrack are not merely a detector and a tracker within this system. They are the core modules that generate object-level geometry and motion semantics throughout the full data flow. FAST-LIO provides a stable ego pose, PointPillars provides temporally ordered 3D detections, and MCTrack turns those detections into track states that can be propagated over time. Only then do dynamic weighting and the joint backend become practically realizable.

4 Engineering Implementation and System Integration

4.1 Overall Code Organization

The project organizes its code in three layers: `Scripts/`, `catkin_ws/src/ME5400/scripts/`, and `catkin_ws/src/fast_lio/`. The top-level scripts handle one-click startup and experiment scheduling. The ME5400 package handles ROS-side bridging, tracking, the backend, and recording. The FAST-LIO package handles the localization front-end and the dynamic-weight logic. Since this report focuses more on system structure than on repository details, Table 2 summarizes the representative entry scripts and their functions instead of enumerating the entire directory tree.

Table 2: Representative entry scripts of the online system and their functions

Module	Representative entry scripts	Function summary
Online baseline	<code>run_baseline.sh</code> <code>run_fastlio.sh</code>	Executes the pure FAST-LIO online baseline pipeline and records the baseline trajectory in <code>trajectory_baseline.txt</code> .
Online optimization	<code>run_optimized.sh</code> <code>run_pointpillars_node.sh</code> <code>run_mctrack_online_node.sh</code> <code>run_joint_backend_ego.sh</code>	Orchestrates PointPillars, MCTrack, dynamic-weight FAST-LIO, and the joint backend, and outputs the optimized front-end trajectory <code>trajectory.txt</code> together with the backend trajectory <code>trajectory_joint.txt</code> .
One-click online comparison	<code>run_all.sh</code>	Sequentially runs the online baseline and online optimization pipelines and archives the complete runtime logs into the result directory.
Evaluation and archiving	<code>evaluate_trajectories.py</code> <code>odom_to_tum_recorder.py</code>	Uniformly generates quantitative metrics and visualization files, including <code>metrics.json</code> , trajectory-error comparison plots, and ablation summary figures.

4.2 Online System Design

In online mode, `run_all.sh` sequentially executes `run_baseline.sh` and `run_optimized.sh`. The former only launches `roscore`, FAST-LIO, and `rosbag` playback to produce a pure localization baseline. The latter additionally launches PointPillars, MCTrack, FAST-LIO in both mode, and the joint backend, while `odom_to_tum_recorder.py` records both the optimized front-end trajectory and the backend trajectory. This dual-track design guarantees a fair comparison under the same bag, the same environment, and the same evaluation script.

The control flow of the online optimization script is clear. PointPillars starts first. MCTrack starts next and waits until it is ready to receive poses and detections. FAST-LIO with dynamic weighting is then launched. Finally, the joint backend starts and records `/joint_backend/odom`. Figure 5 shows the true execution-level temporal logic of this chain.

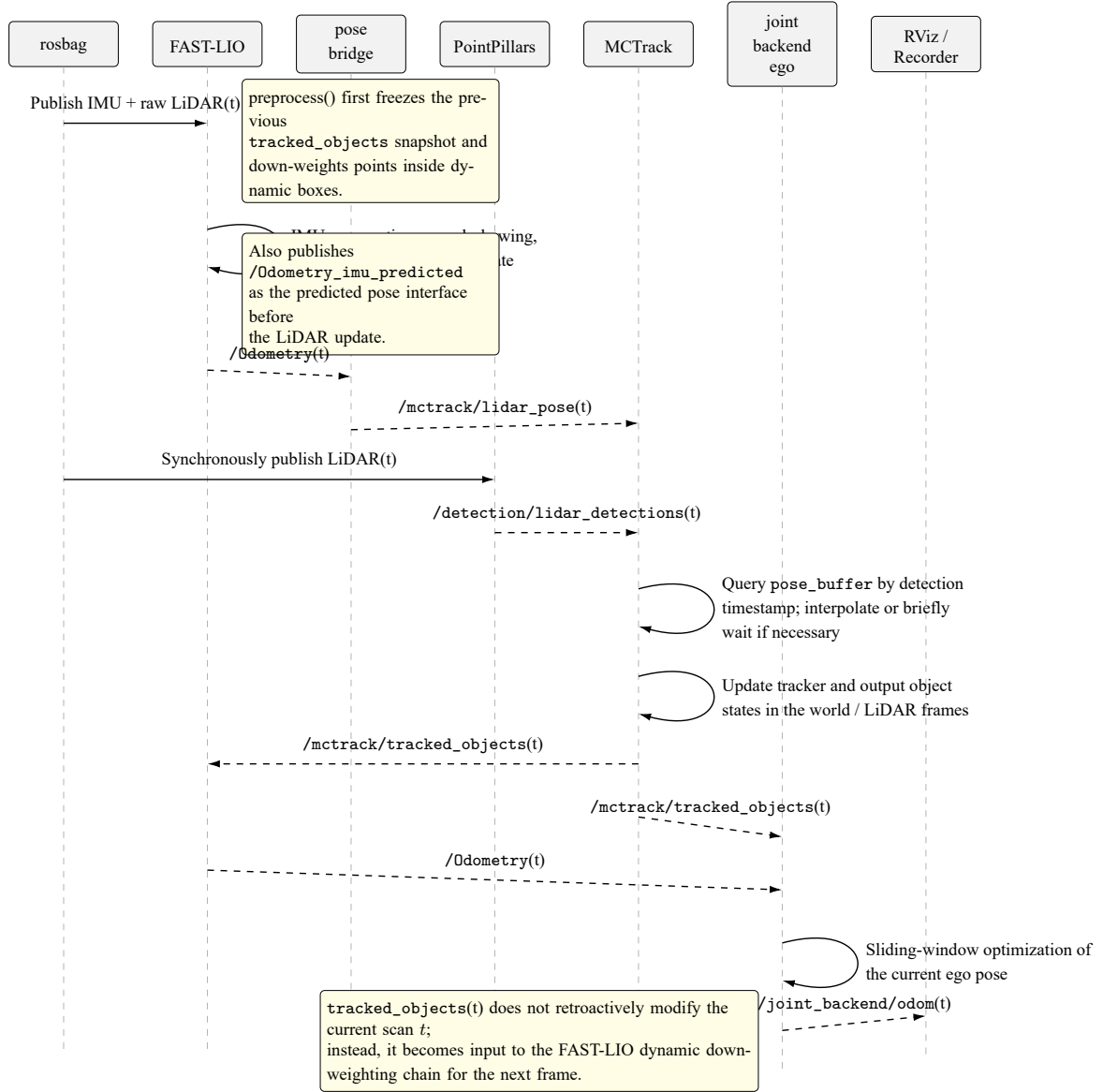


Figure 5: Execution-level temporal logic of the online optimization chain. When FAST-LIO processes the current LiDAR frame, it uses the frozen `tracked_objects` snapshot from the previous frame. The tracking result produced by MCTrack at the current frame mainly enters the dynamic down-weighting chain in the next frame.

As Figure 5 shows, the online chain is not a synchronous loop in which the current-frame detections immediately modify the current-frame point cloud. Instead, it is an asynchronous loop with a one-frame delay. During `preprocess()`, FAST-LIO first freezes the previous `/mctrack/tracked_objects` snapshot and softly down-weights points in the current scan that fall inside dynamic boxes. The current `/Odometry(t)` is then passed through the bridge to MC-Track, aligned with `/detection/lidar_detections(t)` from PointPillars, and used to update object states. The newly produced `/mctrack/tracked_objects(t)` is then sent to both FAST-LIO and the joint backend, but its effect on FAST-LIO appears only in the next frame. This design avoids intra-frame blocking and explains why the current system can form a front-end closed loop while remaining runnable online.

After rosbag playback finishes, the script archives `trajectory.txt`, `trajectory_joint.txt`, and all figures into `Results/<seq>_results/Online/`. The advantage of this code design is that dependencies are explicit, startup order is fixed, and result directories are standardized. Its limitation is that the system is still loosely coupled at the topic level: modules share messages rather than a unified graph state.

4.3 Result Archiving and the Reproducible Experimental Pipeline

The project ultimately organizes key files under `Results/<seq>_results/Online/`. The naming rules for trajectory files, log files, image files, and `metrics.json` are uniform, which makes report writing and later batch analysis straightforward. For a course project, being runnable and being reproducibly evaluable are equally important. At the code-design level, the project’s strongest advantage is precisely that it has formed a complete online experimental loop.

5 Dynamic Weight Optimization

5.1 Design Motivation and Code Location

The core idea of dynamic-weight optimization is to stop treating points inside detected boxes as a binary keep-or-remove decision. Instead, their contribution is continuously reduced according to the object’s motion state and confidence. This decreases the contamination caused by dynamic objects while avoiding the complete loss of local geometry that accompanies hard removal. The current implementation is mainly located in `catkin_ws/src/fast_lio/src/preprocess.cpp`. After FAST-LIO receives `/mctrack/tracked_object`, each object is converted into an internal `DynamicObject`, which stores the object center, bounding-box half-sizes, local yaw angle, timestamp, and weight.

Compared with earlier versions, the default parameters for dynamic weighting in the cur-

rent code are explicitly fixed as follows:

- reference velocity $v_{\text{ref}} = 10.0$;
- reference acceleration $a_{\text{ref}} = 4.0$;
- velocity penalty coefficient $\alpha_v = 0.5$;
- acceleration penalty coefficient $\alpha_a = 0.3$;
- detection-score penalty coefficient $\alpha_s = 0.2$;
- minimum weight $w_{\text{min}} = 0.2$;
- bounding-box margin $xy = 0.5 \text{ m}$, $z = 0.5 \text{ m}$;
- object timeout 1.0 s, with a minimum trajectory-length threshold of 3.

5.2 Weight Computation

Suppose that a tracked target has velocity v , acceleration a , and detection score s . The code first normalizes these quantities:

$$\hat{v} = \text{clamp}\left(\frac{v}{v_{\text{ref}}}\right), \quad \hat{a} = \text{clamp}\left(\frac{a}{a_{\text{ref}}}\right), \quad \hat{s} = \text{clamp}(s). \quad (12)$$

It then constructs the penalty term

$$p = \alpha_v \hat{v} + \alpha_a \hat{a} + \alpha_s \hat{s}, \quad (13)$$

and computes the dynamic weight as

$$w = \text{clip}(1 - p, w_{\text{min}}, 1). \quad (14)$$

This means that the faster the target, the larger its acceleration, and the higher its detection score, the more the system believes it is a truly dynamic object, and the smaller the point weight it assigns. Conversely, if a trajectory is short, the speed is close to zero, or the detection score is not high, the system tends to preserve a larger point weight, thereby avoiding over-penalizing static or unreliable objects.

5.3 How the Weight Is Injected into FAST-LIO

The weight becomes effective in point-cloud preprocessing. For each input point, the system calls `computePointWeight()` to determine whether the point falls inside any dynamic-object

bounding box. If it does, the point’s `intensity` is updated to the smaller of the original intensity and the dynamic weight. The benefit of this design is that it does not require rewriting the main FAST-LIO data structures; the existing point representation is reused to propagate weights through the later mapping and residual-computation chain.

Compared with outright deleting dynamic points, this soft down-weighting strategy has two direct advantages. First, it is more robust to box-boundary error, detection jitter, and partially dynamic regions; a single biased detection does not cause all corresponding points to be discarded. Second, it allows the system to modulate the influence of points continuously according to object motion attributes, rather than reducing the problem to a black-or-white binary rule. The current experiments also indicate that the truly stable gain comes mainly from the combination of this module with MCTrack, rather than from the backend.

6 Joint Backend Optimization

6.1 The Ego-Only Sliding-Window Problem

The joint backend is implemented in `catkin_ws/src/ME5400/scripts/joint_backend_ego_node.py`, with configuration stored in `catkin_ws/src/ME5400/config/joint_backend_ego.yaml`. Its design goal is to validate, at relatively low cost, whether object information can help ego localization. It optimizes only the ego state and does not optimize object states. The key configuration parameters in the current YAML file are: a window length of 8 frames, object-factor coefficient $\lambda_{\text{obj}} = 0.12$, score threshold 0.6, track-length threshold 5, maximum allowed timestamp difference 0.1 s, maximum allowed velocity jump 2.0 m/s, minimum object weight 0.03, Huber as the robust kernel, and scale 0.5. If the `/Odometry` covariance is invalid, the system falls back to `[0.20, 0.20, 0.10]`.

The optimization variable for each frame in the window is written as

$$\mathbf{p}_i = [x_i, y_i, \psi_i]^\top, \quad (15)$$

meaning that only planar position and yaw are retained per frame. This design does not merge object states into the unified variable set, so computation remains lightweight, the implementation stays simple, and online execution remains feasible. The trade-off is that object errors cannot be jointly estimated and can only be used as external observations.

6.2 Residual Modeling and the Gating Strategy

The joint backend includes two major residual types. The first is odometry-consistency residuals: relative poses between adjacent FAST-LIO frames are treated as the primary constraints of

the sliding window, ensuring that optimization does not deviate excessively from the original odometry. The second is object-observation consistency residuals: for each frame, the backend matches the nearest tracking snapshot in time and retains only objects that satisfy score, track-length, velocity-jump, and time-synchronization gates. For each retained object, the code first extrapolates its world-coordinate position to the current frame timestamp using its velocity and then projects it into the ego local frame used during optimization, yielding the predicted observation z_{ik}^{pred} . The measured observation z_{ik}^{meas} comes from the LiDAR-local pose stored in `TrackedObject`. The object residual is therefore

$$r_{ik}^{\text{obj}} = z_{ik}^{\text{pred}} - z_{ik}^{\text{meas}}. \quad (16)$$

The overall optimization objective is

$$\min_{\{\mathbf{p}_i\}} \sum_{i=2}^N \|r_i^{\text{odom}}\|_{W_i}^2 + \sum_{i=1}^N \sum_{k \in \mathcal{V}_i} w_{ik} \rho\left(\|r_{ik}^{\text{obj}}\|_2^2\right), \quad (17)$$

where $\rho(\cdot)$ is a Huber or Cauchy robust kernel and $\mathcal{V}_i$ denotes the set of objects that pass the gates. In the current code, the object-factor weight is approximately written as

$$w_{ik} = \max\left(w_{\min}^{\text{obj}}, \sqrt{\lambda_{\text{obj}}} \cdot (0.5 \hat{s}_{ik} + 0.5 \hat{\ell}_{ik})\right), \quad (18)$$

that is, it is jointly determined by normalized score and normalized trajectory length.

6.3 Value and Limitations of the Joint Backend

The engineering value of this backend is that it validates a key proposition: even when object states are not treated as explicit optimization variables, stable tracked objects can still act as temporary moving anchors that impose additional constraints on the ego trajectory. In other words, the system has already moved beyond the view that dynamic targets are merely noise to be filtered out.

Its limitations, however, are equally clear. Because object states come from point estimates of the tracker and are extrapolated through velocity propagation, any noise in object velocity, detection boxes, or association results can be injected back into the ego pose. In the current experiments, the joint backend does reduce global drift on some sequences, but it also degrades performance noticeably on others. Therefore, the conclusion of this section is not that the joint backend is already mature, but rather that the direction is valid while the current ego-only implementation remains an engineering prototype that still needs substantial refinement.

7 Experimental Analysis

7.1 Experimental Setup and Result Sources

The experiments in this report are uniformly based on the currently archived online results under Results/. The online experiments cover seven sequences: 0003, 0006, 0007, 0009, 0010, 0013, and 0020. All numerical values are taken from the corresponding metrics.json files, while the figures correspond to evaluation_result.png, evaluation_joint_backend.png, and ablation_bar.png.

In online mode, three configurations are compared: Baseline, FAST-LIO+MCTrack, and Joint Backend. The evaluation metrics include ATE RMSE, RPE RMSE, KITTI relative translation error Tr , and relative rotation error Rot .

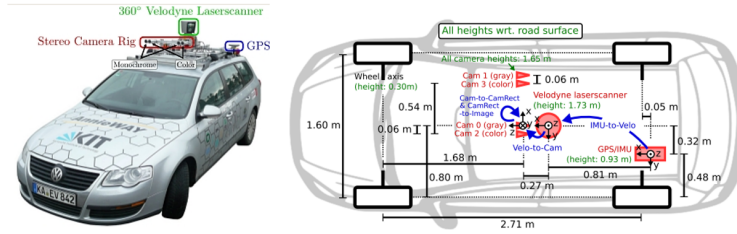


Figure 6: KITTI data-collection platform. All experiments in this report are based on archived KITTI Tracking results.

7.2 Online Experiments: Aggregate Results

Table 3 summarizes ATE and RPE for all online sequences. It can be seen that FAST-LIO+MCTrack outperforms the Baseline on all seven sequences, indicating that the most stable gain in the current system comes from the front-end closed loop. In contrast, Joint Backend further reduces ATE on only four of the seven sequences, indicating clearly insufficient stability.

Table 3: Summary of online ATE/RPE results. Improvement percentages are computed relative to Baseline; positive values indicate lower error.

Sequence	Baseline ATE	F+M ATE	F+M Improvement	Joint ATE	Joint Improvement	RPE RMSE (B/F/J)
0003	0.5294	0.4513	+14.75%	0.5875	-10.98%	0.2049 / 0.1809 / 0.2393
0006	0.5238	0.4710	+10.09%	0.5576	-6.45%	0.1324 / 0.0869 / 0.1677
0007	2.3128	2.2393	+3.18%	2.2393	+3.18%	0.1716 / 0.1683 / 0.1683
0009	1.6831	1.6675	+0.93%	3.3082	-96.56%	0.1787 / 0.1651 / 1.9514
0010	3.7651	0.8680	+76.95%	0.9632	+74.42%	0.2774 / 0.2281 / 0.2334
0013	0.6047	0.5816	+3.82%	0.5816	+3.82%	0.1495 / 0.1392 / 0.1392
0020	11.7323	6.7938	+42.09%	6.8829	+41.33%	0.2250 / 0.2245 / 0.2270

Table 4 reports KITTI relative translation and rotation errors in online mode. Across the seven sequences with sufficient segment length, FAST-LIO+MCTrack outperforms the baseline

on six sequences, whereas Joint Backend outperforms the baseline on only three. This indicates that the current backend behaves more like an occasional global-drift corrector than a consistently reliable local refinement module.

Table 4: Summary of online KITTI relative errors. Rot is given in deg/100m, with the order Baseline / F+M / Joint.

Sequence	Baseline Tr(%)	F+M Tr(%)	F+M Improvement	Joint Tr(%)	Joint Improvement	Rot (B/F/J)
0003	1.4794	1.3516	+8.63%	1.6856	-13.94%	1.1509 / 1.1569 / 1.3699
0006	4.1614	4.0605	+2.42%	4.3990	-5.71%	12.8416 / 11.6291 / 16.0675
0007	3.9042	3.9267	-0.57%	3.9267	-0.57%	4.9702 / 5.0234 / 5.0234
0009	2.5630	2.5281	+1.36%	4.2200	-64.65%	3.2425 / 3.2294 / 3.7550
0010	1.3884	0.9773	+29.61%	1.1140	+19.76%	1.1274 / 1.1368 / 1.5713
0013	2.4374	2.3741	+2.60%	2.3741	+2.60%	2.3026 / 2.2419 / 2.2419
0020	3.2245	2.3967	+25.67%	2.4302	+24.63%	1.1894 / 1.1748 / 1.2326

Among the online results, the two strongest success cases are Sequences 0010 and 0020. For Sequence 0010, ATE RMSE decreases dramatically from 3.7651 m to 0.8680 m, an improvement of 76.95%. For Sequence 0020, ATE RMSE drops from 11.7323 m to 6.7938 m, an improvement of 42.09%. Together, these two cases show that when baseline drift is substantial and dynamic objects are highly involved, dynamic-point down-weighting together with stable tracking can significantly improve localization. Figure 7 further presents the trajectory evaluation and dynamic-vehicle tracking visualization for Sequence 0020.

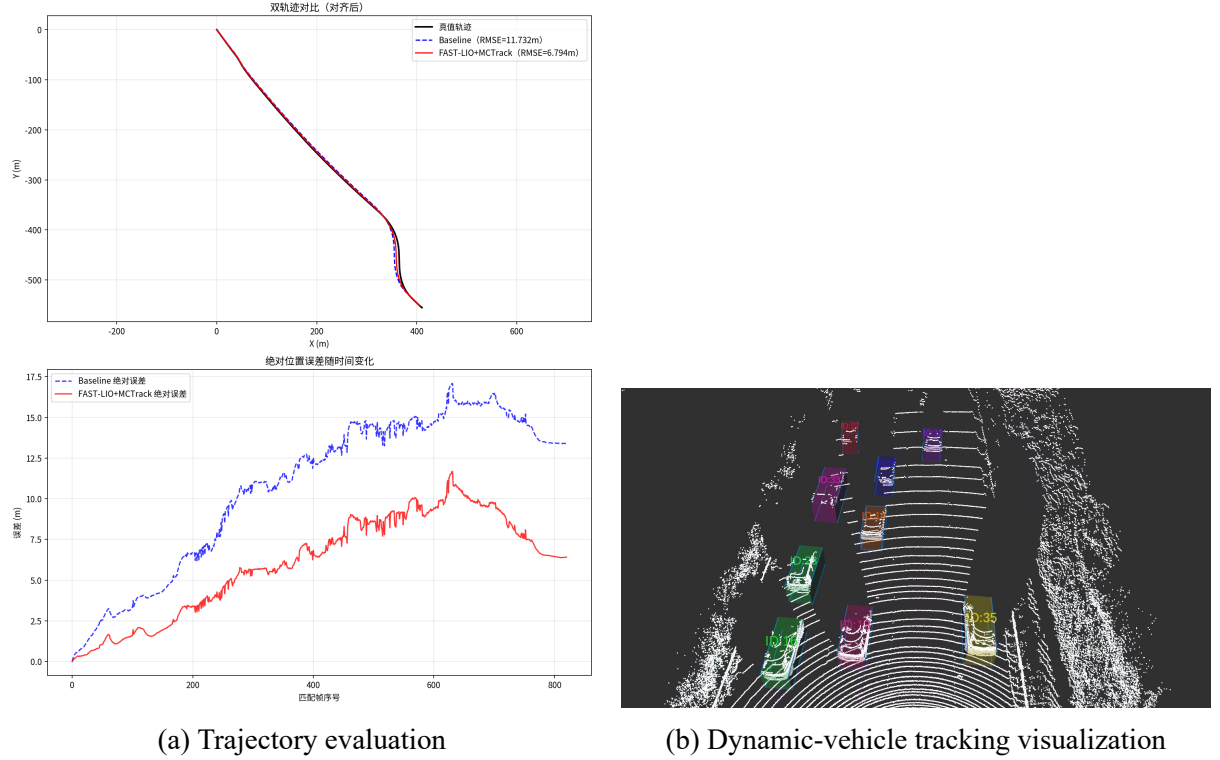


Figure 7: Representative online results for Sequence 0020. The left panel shows that the red FAST-LIO+MCTrack trajectory is clearly closer to the ground truth than the blue baseline. The right panel visualizes dynamic-vehicle tracking on the same sequence.

7.3 Integrated Analysis and Failure Modes

Three relatively clear conclusions can be drawn from the full set of current results.

First, the truly stable gain comes from the front-end closed loop. Across the seven online sequences, FAST-LIO+MCTrack outperforms the baseline on 7/7 sequences. For KITTI relative translation error, improvements are observed on 6/7 evaluable sequences. This shows that the most mature and trustworthy module in the project is the closed loop of pose-assisted tracking and tracking-fed dynamic weighting.

Second, the joint backend is still fragile. Among the currently archived results, the representative failure cases are no longer the old Sequence 0011, but Sequences 0009 and 0012. In online mode for Sequence 0009, the ATE RMSE of Joint Backend degrades from 1.6831 m to 3.3082 m. For Sequence 0012, the corresponding degradation is from 0.0099 m to 0.0606 m. These cases show that once velocity propagation, temporal alignment, or gating conditions are even slightly biased, the ego-only backend can re-inject noise into the ego trajectory.

Third, the current backend behaves more like a long-horizon drift corrector than a general-purpose local refinement module. Using online Sequence 0020 as an example, the joint backend is more effective at maintaining or partially strengthening global-trajectory improvement than at consistently reducing frame-wise local RPE. In other words, the experimental conclusion here is not that the joint backend is finished, but that the direction is valid while the current implementation still requires stronger activation strategies and more robust object modeling.

8 Conclusion and Future Plan

This report reorganizes the entire project according to the requested seven-chapter structure, restoring the original background, system design, engineering implementation, dynamic-weight details, and joint-backend details into the main text, while also incorporating all current experimental results and figures from the repository into one unified document. At the system level, the central conclusion is clear: the project has successfully built a bidirectional collaborative closed loop around FAST-LIO, PointPillars, and MCTrack, and the most stable and reliable gain comes from the front-end dynamic-weight loop rather than from the current ego-only joint backend.

Future work should concentrate on three main directions. First, an adaptive activation mechanism should be added to the joint backend so that object constraints are enabled only when tracking is stable, enough targets are available, and the baseline clearly suffers from drift. Second, object states should be brought into the unified optimization variables instead of remaining outside an ego-only formulation. Third, the system should move from object-box-level constraints toward rigid-body surface constraints so that dynamic vehicles are genuinely

transformed from disturbance sources into moving landmarks. If these directions continue to be developed, the project can evolve from a course-level engineering closed-loop system into an object-centric LiDAR-inertial joint optimization framework with stronger research contribution.

References

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2: Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. CVPR*, 2019, pp. 12697–12705.
- [3] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. CVPR*, 2012, pp. 3354–3361.
- [4] X. Weng, J. Wang, D. Held, and K. Kitani, “3D multi-object tracking: A baseline and new evaluation metrics,” in *Proc. IROS*, 2020, pp. 10359–10366.
- [5] S. Fang and H. Li, “LiDAR SLAMMOT based on Confidence-guided Data Association,” *arXiv preprint arXiv:2412.01041*, 2024.
- [6] Z. Zhu, J. Zhao, K. Huang, X. Tian, J. Lin, and C. Ye, “LIMOT: A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6600–6607, 2024.
- [7] P. Tian and H. Li, “Visual SLAMMOT Considering Multiple Motion Models,” *arXiv preprint arXiv:2411.19134*, 2024.
- [8] J. Morris, Y. Wang, and V. Ila, “Online Dynamic SLAM with Incremental Smoothing and Mapping,” *arXiv preprint arXiv:2509.08197*, 2025.
- [9] X. Tian, Z. Zhu, J. Zhao, G. Tian, and C. Ye, “DL-SLOT: Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1017–1027, 2024.

Acknowledgment

As I complete this report for the ME5400 project, I would like to express my sincere gratitude to everyone who has supported me throughout this work. This report is not only a summary of the project itself, but also a meaningful record of my learning, research, and engineering practice during my study.

First of all, I would like to thank all the teachers in the College of Design and Engineering at the National University of Singapore for their teaching, guidance, and academic support during my master's study. Their patience and dedication have greatly deepened my understanding of both research and engineering practice.

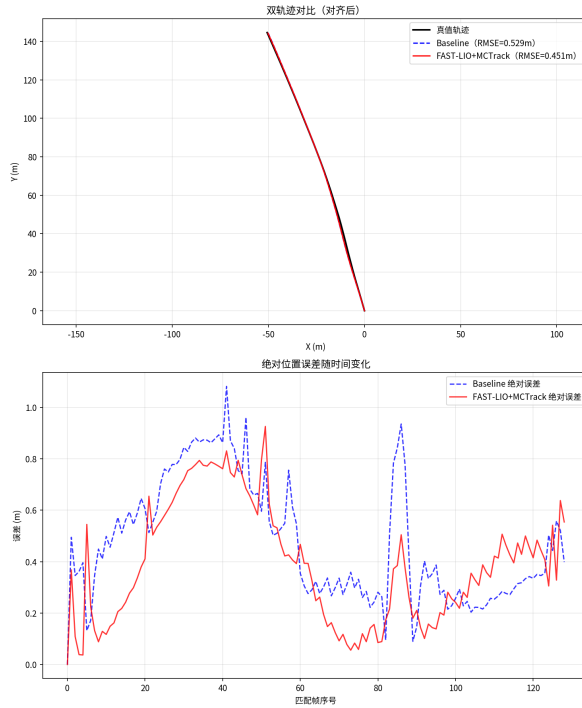
I am especially grateful to my supervisor, Professor Marcelo H. Ang Jr., for his valuable guidance and support throughout this project. I would also like to sincerely thank Professor Teo Chee Leong for his time and thoughtful evaluation of this work. My heartfelt thanks also go to Han Yuhang, whose guidance was of great help during this research, and to every member of our research group for their companionship, discussions, and encouragement.

Finally, I would like to express my deepest respect and gratitude to my family and friends. Their love, trust, and support have always given me strength, especially during difficult and uncertain moments.

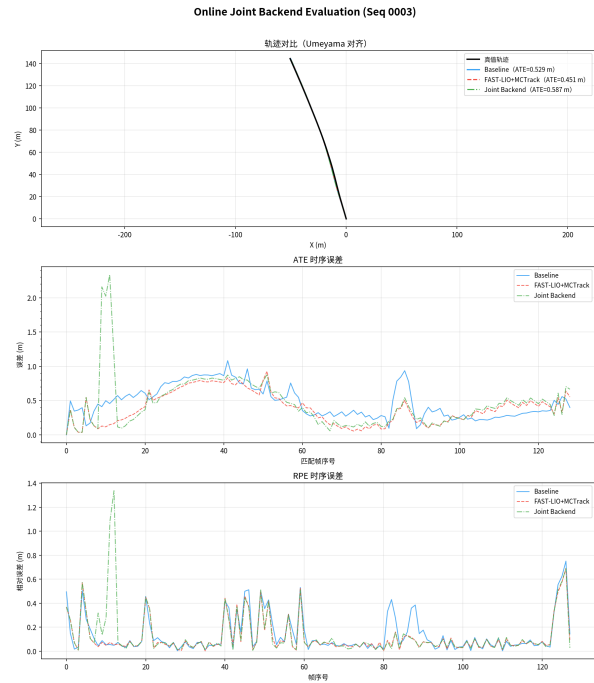
As I move forward into future work and life beyond school, I will carry this kindness with me and continue my journey with sincerity, gratitude, and hope.

Appendix: All Online Experimental Figures

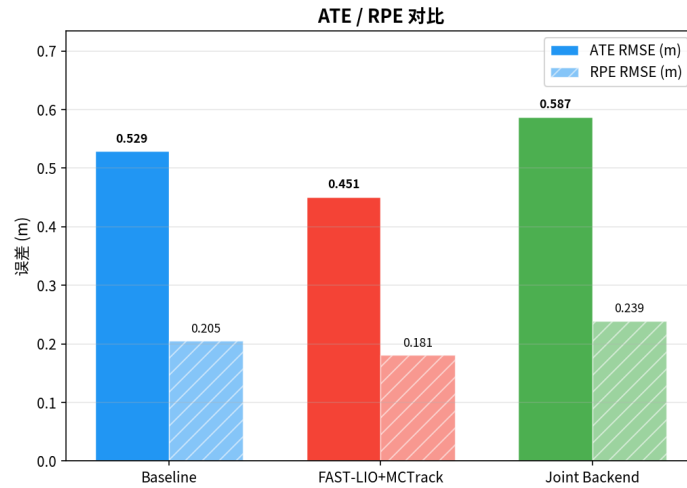
This appendix inserts all online result figures by sequence. Each sequence includes the online front-end comparison, the joint-backend evaluation, and the bar-chart comparison.



(a) Online front-end comparison

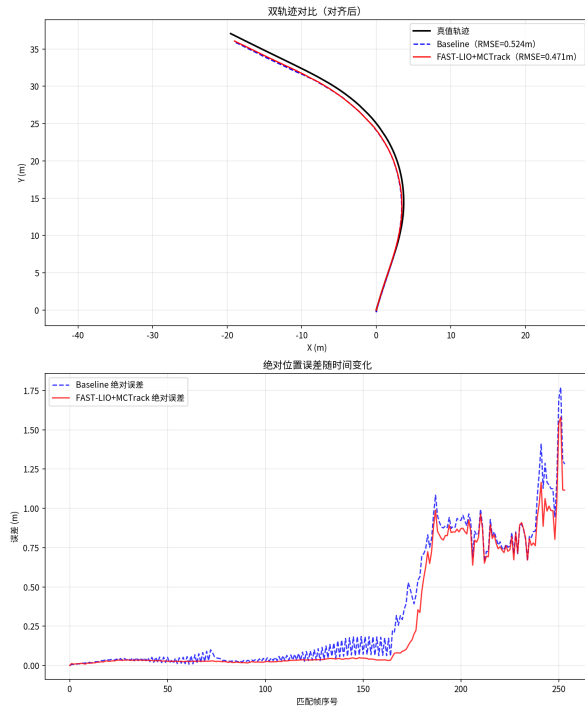


(b) Joint backend evaluation

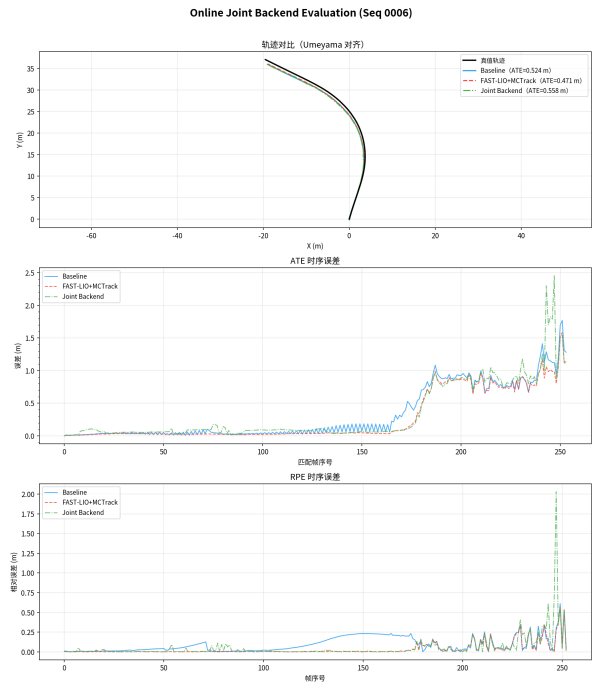


(c) ATE/RPE bar comparison

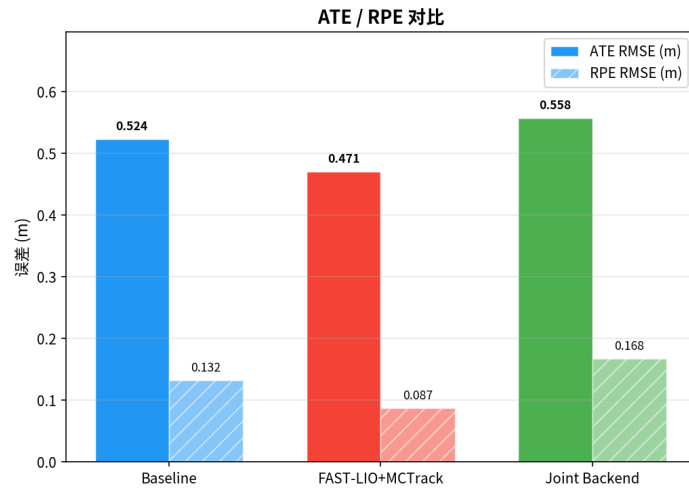
Figure 8: Complete online experimental results for Sequence 0003.



(a) Online front-end comparison

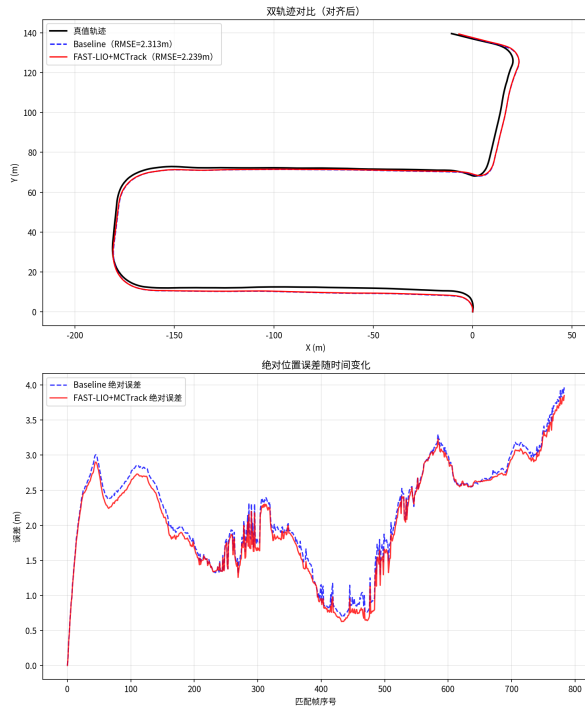


(b) Joint backend evaluation

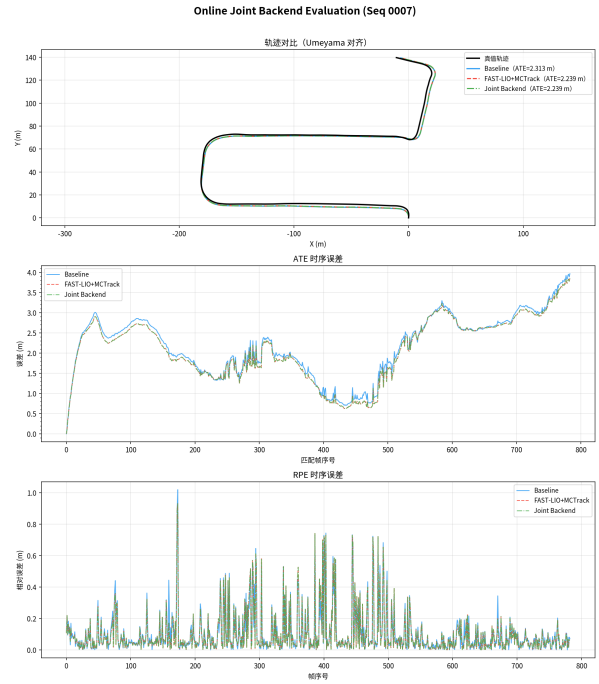


(c) ATE/RPE bar comparison

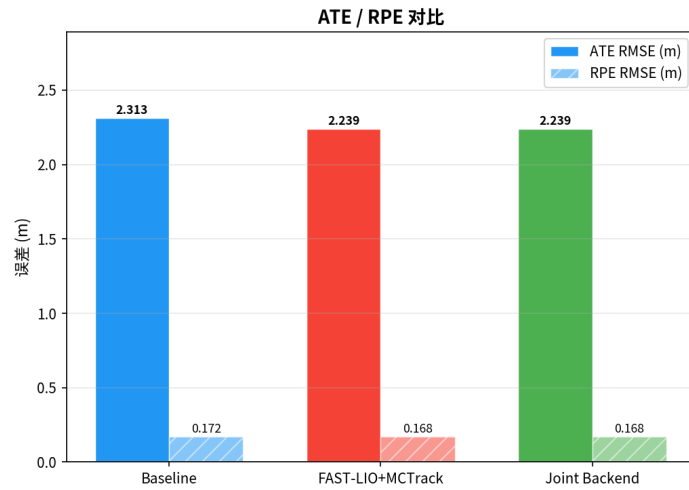
Figure 9: Complete online experimental results for Sequence 0006.



(a) Online front-end comparison

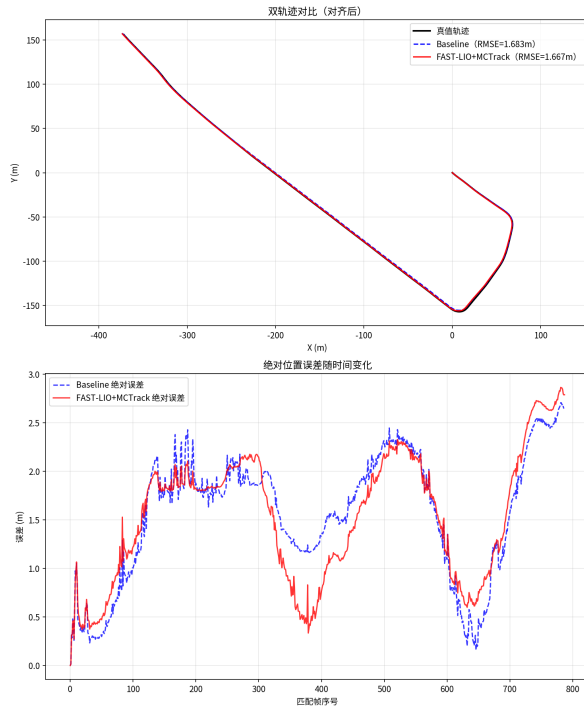


(b) Joint backend evaluation

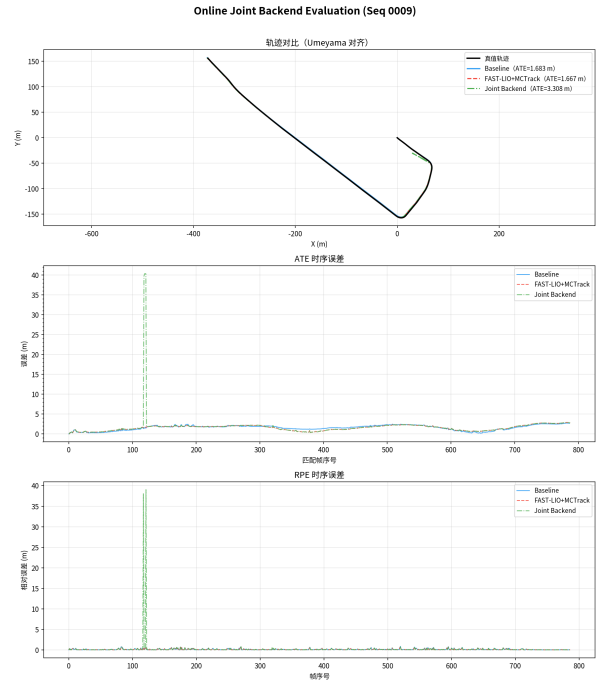


(c) ATE/RPE bar comparison

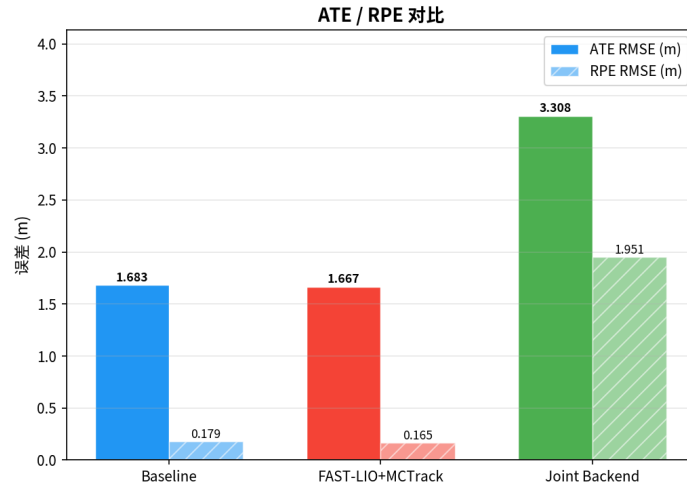
Figure 10: Complete online experimental results for Sequence 0007.



(a) Online front-end comparison

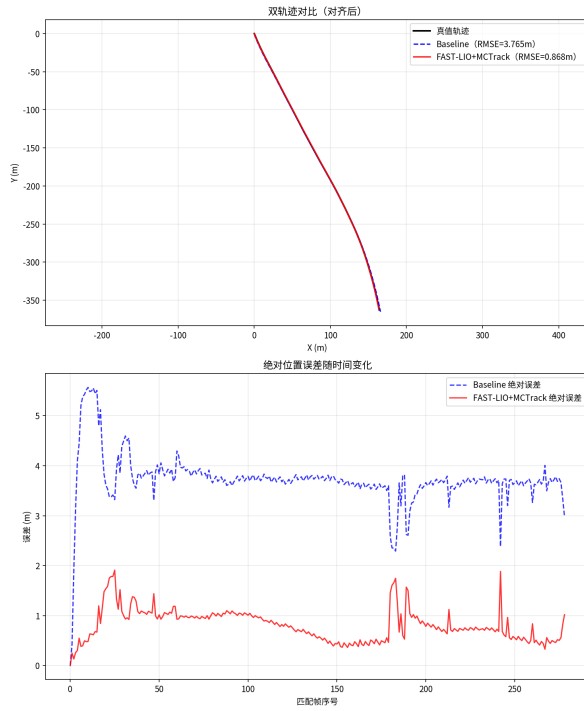


(b) Joint backend evaluation

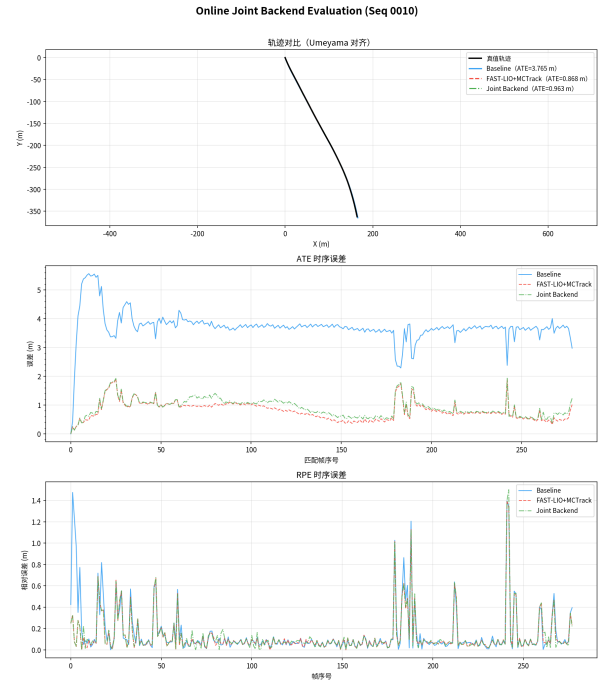


(c) ATE/RPE bar comparison

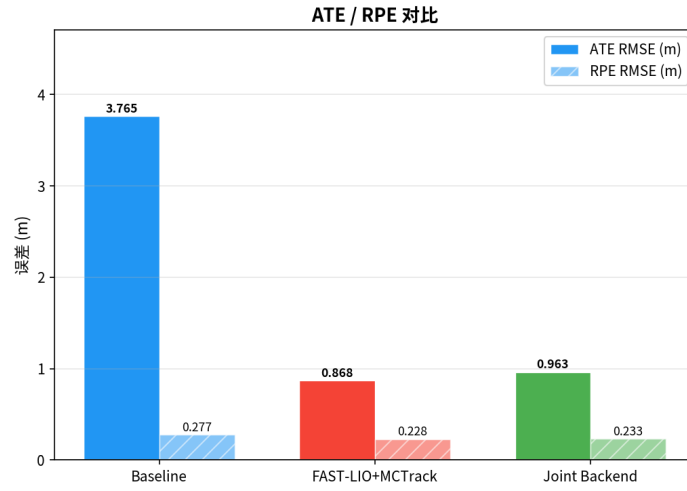
Figure 11: Complete online experimental results for Sequence 0009.



(a) Online front-end comparison

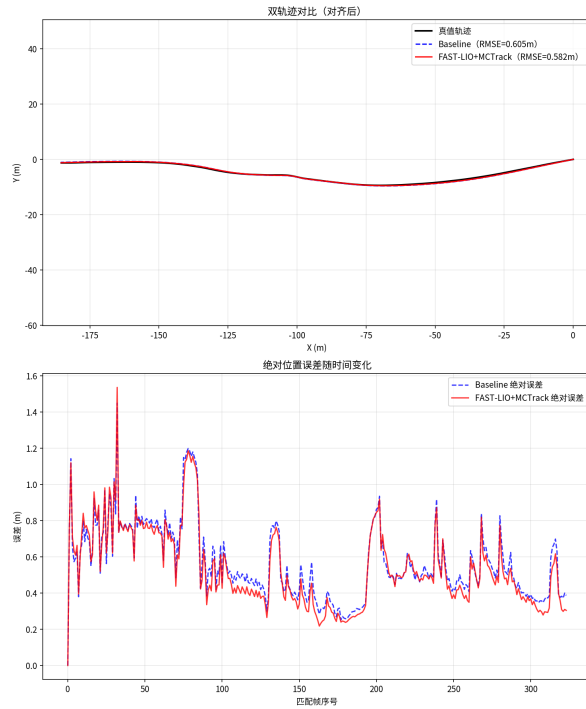


(b) Joint backend evaluation

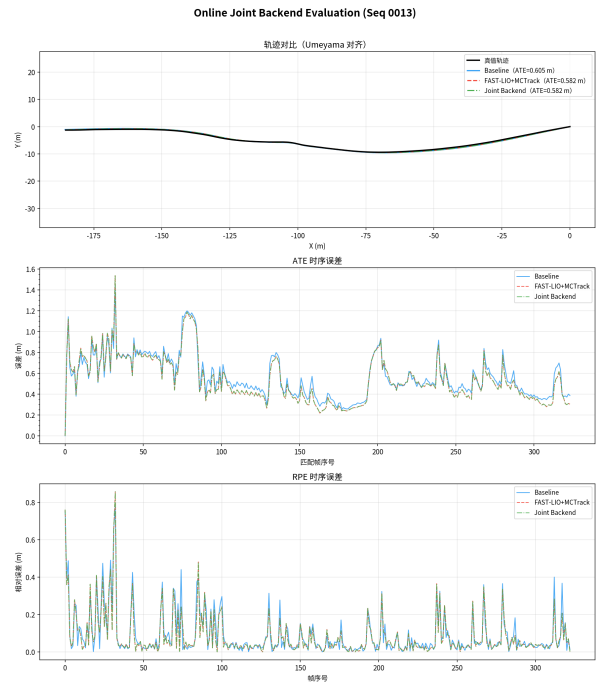


(c) ATE/RPE bar comparison

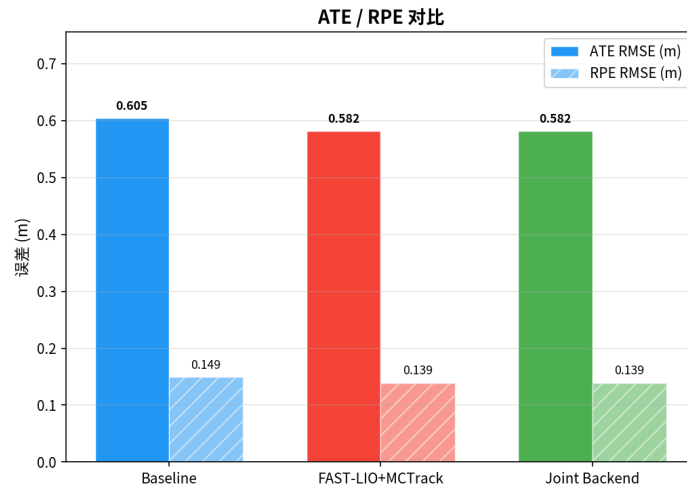
Figure 12: Complete online experimental results for Sequence 0010.



(a) Online front-end comparison

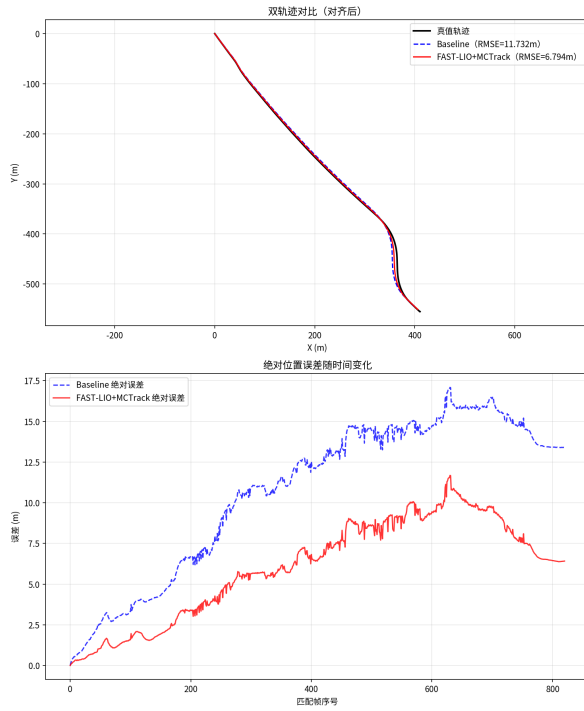


(b) Joint backend evaluation

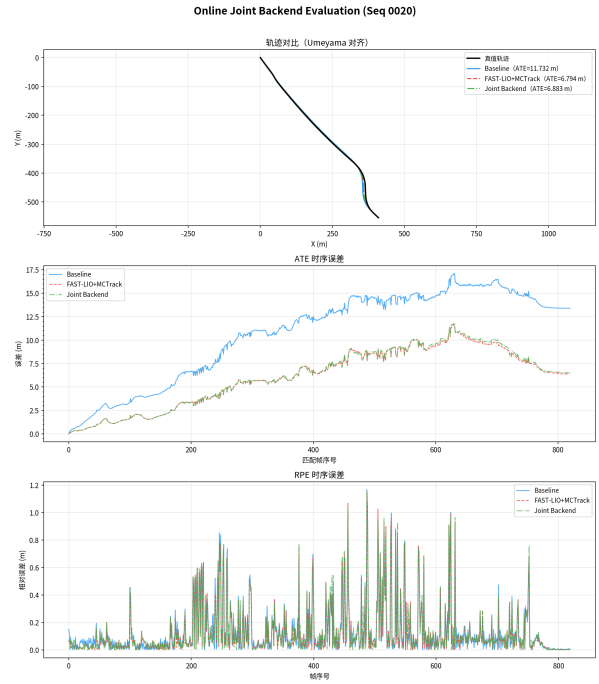


(c) ATE/RPE bar comparison

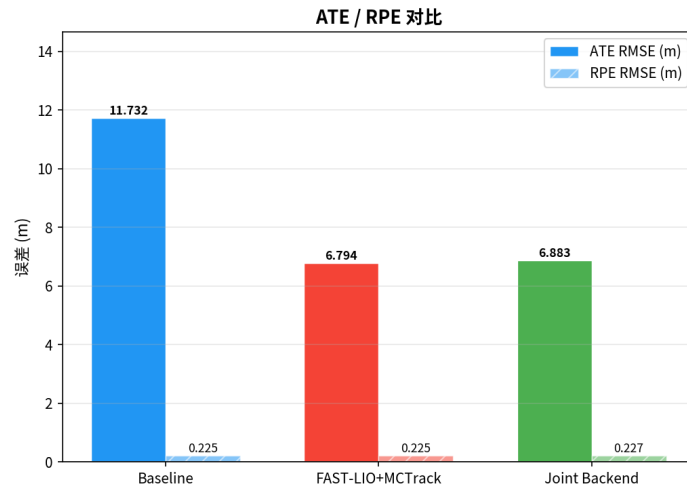
Figure 13: Complete online experimental results for Sequence 0013.



(a) Online front-end comparison



(b) Joint backend evaluation



(c) ATE/RPE bar comparison

Figure 14: Complete online experimental results for Sequence 0020.